

High Throughput Large Integer Multipliers Built on Optimized FPGA DSP Block Primitives

Abstract—Large integer multiplication is critical in cryptographic applications such as Fully Homomorphic Encryption (FHE) and RSA where it represents the main step in computing the modular multiplication used to generate keys. Most previous work building such multipliers on FPGAs has overlooked the importance of considering the underlying architecture of the Digital Signal Processing (DSP) Blocks present in modern FPGAs. These DSP blocks can operate at high frequency but present an irregular input size that must be tiled to create larger multipliers, and have complex pipelining configurations. In this paper, we show how an efficient base multiplier can be built using modern DSP blocks, then tiled to large sizes efficiently through an automated architecture-aware tiling and fast pipelined datapath generation, while maintaining resource efficiency and high throughput. We demonstrate the automation of tiling, and establish a new performance standard for large integer multipliers on FPGAs.

I. INTRODUCTION

Multiplication should be simple. However, as the operands increase in size, the complexity of creating multipliers that can be mapped efficiently to hardware while meeting performance goals increases significantly. Large integer multiplication is specially complex and is widely used in various applications such as cryptography. For instance, Elliptic Curve Cryptography (ECC) often uses bitwidths of 160, 256, 384, and 512 while RSA typically employs even larger sizes such as 1024, 2048, and 4096 [1]. That is, these large wordlengths exceed the capabilities of hardware resources. Squaring a large integer is also a common operation in many applications; for example, in RSA, approximately 22.2% of the operations are squaring operation [2]. Squaring is the specific instance of multiplication where the multiplier and the multiplicand are identical. Although squaring can be performed using general multiplication hardware, specialized squarers exploiting this property can offer significant advantages in terms of resource efficiency.

Field-Programmable Gate Arrays (FPGAs) offer a flexible architecture that can be used to implement a variety of applications in domains such as image processing [3]–[5], digital signal processing [6]–[8], and cryptography [9]–[14]. Since multiplications are so prevalent across different applications, and would consume large numbers of the LUTs used for mapping general functions on FPGAs, DSP blocks have gained significant importance in providing high computational

throughput. They have evolved to become more functional and flexible, offering high throughput and efficiency and lower silicon area than what can be achieved with LUTs. DSP blocks are not only used for multiplication but also for other arithmetic operations such as addition, accumulation, and multiply-accumulate (MAC) operations. However, making the best use of both DSP blocks and LUTs for complex arithmetic implementations remains challenging, as mapping tools do not produce optimal mappings.

In order to multiply large numbers using smaller multipliers, it is necessary to decompose them into smaller pieces that can be mapped to the wordlengths of the underlying architecture. The Schoolbook decomposition is a simple scheme that uses shift and add operations. Comba multiplication [15] reorders partial products for further optimization. Further efficiency can be gained with Karatsuba decomposition [16], which reuses some partial products to reduce the number of multipliers needed, as illustrated in Section II. It is also possible to use the Fast-Fourier Transform (FFT) for extremely large wordlengths [17]–[19]. While traditional FFTs operate on complex numbers, the Number-Theoretic Transform (NTT) is sometimes preferred when performing large integer multiplication and is a special case of the FFT over a finite field Z [20], [21].

Decomposing large multipliers spatially across smaller primitives requires careful consideration. Many decomposition schemes have been proposed and explored mostly for symmetric power-of-2 wordlengths, while the primitives on FPGAs are both asymmetric and non-power-of-2. Furthermore, there are considerations relating to the mix of signed and unsigned parts and how they are mapped to the underlying architecture. DSP blocks also have specific pipelining arrangements that must be respected to achieve high performance. Additionally, the FPGA architecture offers additional possible routes for optimization including the use of the cascade paths between multipliers, that would further enhance mapping. This paper attempts to address this mapping in an architecture-centric manner, enabling small base multipliers and large integer multipliers to be efficiently mapped to FPGAs.

The main contributions of this paper are:

- 1) An architecture-aware base multiplier design that efficiently maps to FPGA DSP blocks while achieving

high frequency and low resource usage.

- 2) A comparative analysis of our base multiplier with an existing vendor 64×64 IP multiplier.
- 3) Automation of the process of building large integer multipliers from the proposed base multiplier, based on analysis on different efficient base multipliers using Schoolbook and Karatsuba decomposition and fast pipelined combinational logic taking into account FPGA architecture features.
- 4) Evaluation of different multiplication sizes using different base sizes and comparing the evaluated performance for 1024×1024 bit multiplier built using HLS, as well as against the approach presented in [22].

Our approach achieved around 55% higher frequency than other approaches while using around 17% fewer DSP blocks.

II. BACKGROUND

A. Decomposition Schemes

Large multipliers are commonly built by decomposing the computation into smaller multipliers that better match the target architecture, trading off multiplier count against shift-and-add overhead.

1) Schoolbook Decomposition

Schoolbook decomposition forms a $2n \times 2n$ product using four $n \times n$ partial products (Equation 1). Let $A = (A_1 \ll n) + A_0$ and $B = (B_1 \ll n) + B_0$.

$$\begin{aligned} A &= (A_1 \ll n) + A_0 \\ B &= (B_1 \ll n) + B_0 \end{aligned}$$

$$\begin{aligned} \text{Schoolbook}(A \times B) &= (A_0 B_0) \\ &+ (A_0 B_1 + A_1 B_0) \ll n \\ &+ (A_1 B_1) \ll 2n \end{aligned} \quad (1)$$

This approach scales quadratically (e.g., nine multipliers for a 3×3 split) and requires shift-and-add logic to combine partial products.

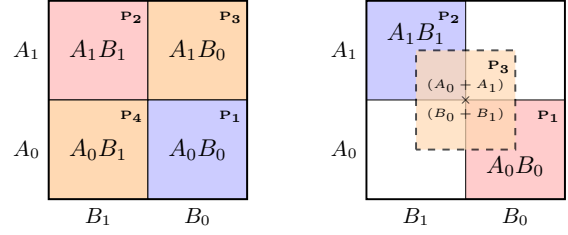
Equation 1 can be re-ordered to save an addition as follows in Equation 2 [22].

$$\begin{aligned} \text{Schoolbook}(A \times B) &= (A_0 B_0 \& A_1 B_1) \\ &+ (A_0 B_1 + A_1 B_0) \ll n \end{aligned} \quad (2)$$

2) Karatsuba Decomposition

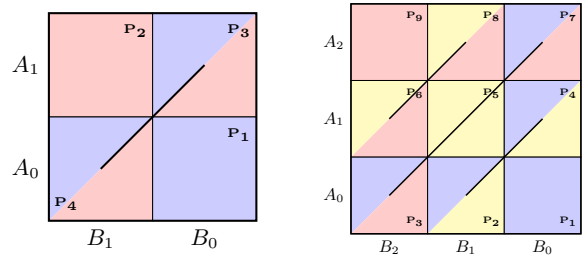
Karatsuba [16] reduces the 2×2 case from four multipliers to three by replacing one multiply with extra additions/subtractions (smaller cost than multiplication). One product is typically $(n+1)$ -bit wide.

$$\begin{aligned} \text{Karatsuba}(A_0 B_1 + A_1 B_0) &= (A_0 + A_1)(B_0 + B_1) \\ &- A_0 B_0 \\ &- A_1 B_1 \end{aligned} \quad (3)$$



(a) Schoolbook decomposition. (b) Karatsuba decomposition.

Fig. 1: Comparison of Schoolbook and Karatsuba decomposition schemes for a $2n \times 2n$ multiplier.



(a) 2×2 Karatsuba decomposition. (b) 3×3 Karatsuba decomposition.

Fig. 2: Optimized Karatsuba decomposition schemes for 2×2 and 3×3 grids. The squares that are two-colored represent the multiplications that can be saved reusing the squares with solid colors.

Figure 1 presents a side-by-side comparison of the Karatsuba and Schoolbook decomposition schemes for a $2n \times 2n$ multiplier.

Karatsuba also extends can be extended larger grids (e.g., 3×3 in Figure 2b), saving 3 multipliers relative to Schoolbook (9 total).

The $(n+1)$ -bit multiply in Karatsuba can be built from an n -bit multiplier plus lightweight logic by splitting each operand into an n -bit part and a 1-bit part (Figure 4). The $n \times 1$ and 1×1 terms reduce to AND gates, and non-overlapping terms can be concatenated (as in Equation 2) to save additions. Using the n -bit multiplier on the LSB side (Figure 4b) reduces the widest adder from $2n+1$ to $n+1$ bits (Figure 3).

B. Tiling Schemes

Having understood how to decompose multiplications, we can now discuss how to tile smaller multipliers spatially to build large ones. The tiling problem is NP-complete [23]–[25], though optimal solutions exist for small sizes (e.g., 64×64 [26]) and many heuristics/ILP approaches target the general case [27]–[31]. A valid tiling covers each bit exactly once (no overlap), trading

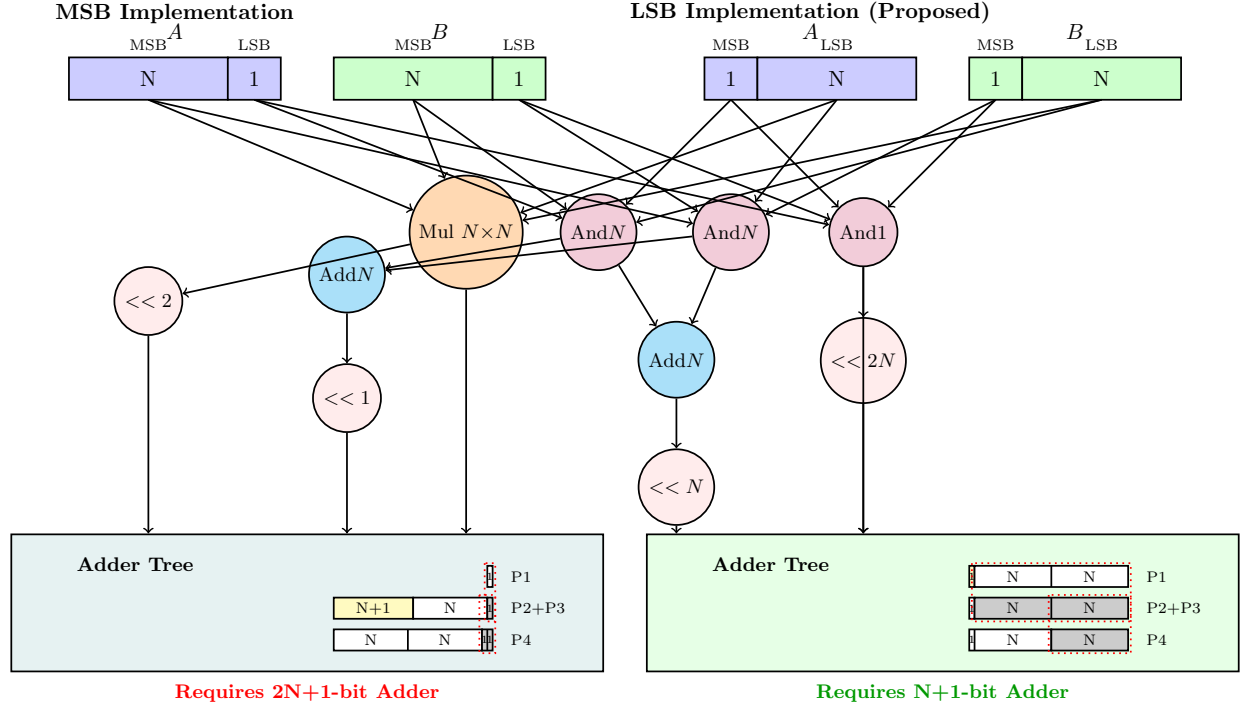


Fig. 3: Side-by-side comparison of $n + 1$ -bit multiplier implementations. The MSB implementation requires a $2n + 1$ -bit adder while the LSB implementation (proposed) only needs an $n + 1$ -bit adder, reducing adder resources by approximately 50%.

flexibility (many small tiles) against fewer shifts/additions (larger tiles).

Tiling can be visualized as a 2-D grid: one operand on x and the other on y , where each tile represents a partial product with a shift determined by the tile position. Figure 5 contrasts 8×8 tilings using sixteen 2×2 tiles (more flexibility, more shift/add) versus four 4×4 tiles (fewer operations). Karatsuba-style reuse can be overlaid on the tiling view (Figure 6) [30]. Practical tiling also reflects device primitives; e.g., UltraScale+ provides 27×18 DSP multipliers, so rectangular tiles matching these dimensions improve DSP utilization.

C. DSP Block Consideration

1) Overview and Applications

A DSP block is a hard FPGA primitive optimized for high-frequency multiply and add/accumulate operations. Modern families expand multiplier and pre/post-adder widths (e.g., the multiplier from 25×18 in Virtex-5/6/7 to 27×18 in UltraScale(+) to 27×24 in Versal) [32]–[35]. DSP blocks are widely used for FIR filtering, sensing, ML inference, and scientific signal processing [36]–[40], and some families also support floating-point modes [41].

2) AMD DSP Slice Architecture

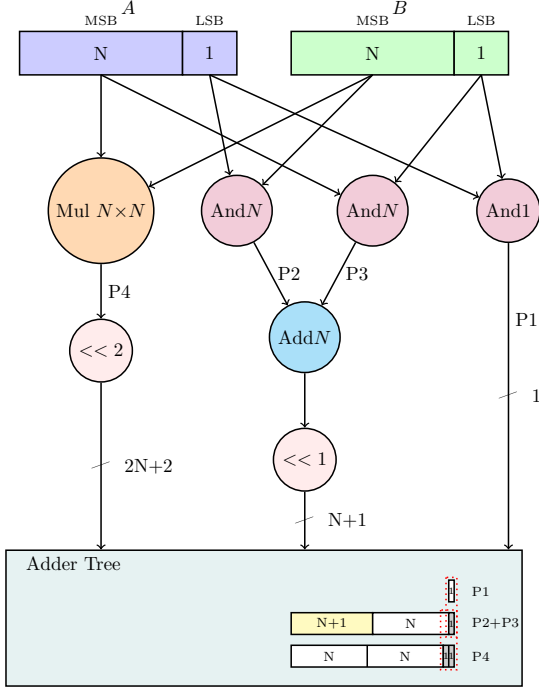
An AMD DSP slice combines a pre-adder, multiplier, multiplexers, and an ALU (Figure 7). Control signals (e.g.,

OPMODE/ALUMODE) select the datapath and ALU expression [42], while optional pipeline registers trade latency for throughput where enabling the four pipeline registers achieves maximum throughput. Cascaded ports (e.g., PCIN/PCOUT) enable efficient column-wise chaining alongside global routing.

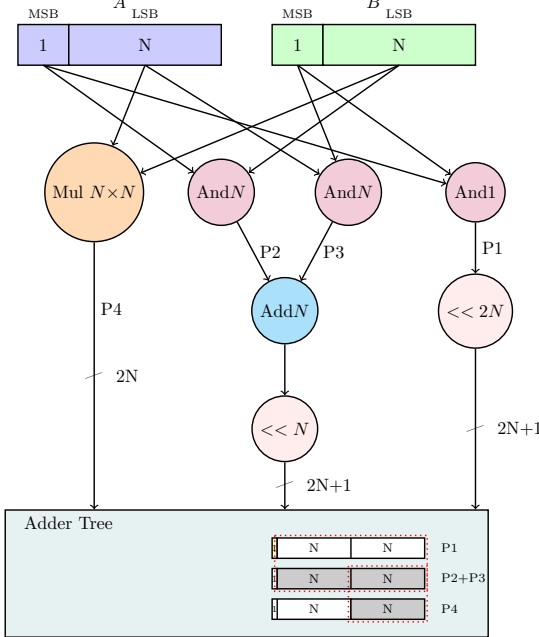
By manipulating control signals, any of the expressions presented in Equation 4 can be implemented using a single DSP slice:

$$DSPOut = \begin{cases} C \pm (B \times (D \pm A) + W + C_{in}) \\ C \pm (A \times (D \pm B) + W + C_{in}) \\ C \pm (B \times (D \pm B) + W + C_{in}) \\ C \pm (A \times (D \pm A) + W + C_{in}) \\ C \pm ((D \pm B)^2 + W + C_{in}) \\ C \pm ((D \pm A)^2 + W + C_{in}) \end{cases} \quad (4)$$

The cascade path routes PCOUT to PCIN to support chained accumulation via the Z multiplexer; OPMODE/ALUMODE select the ALU expression (Equation 5).



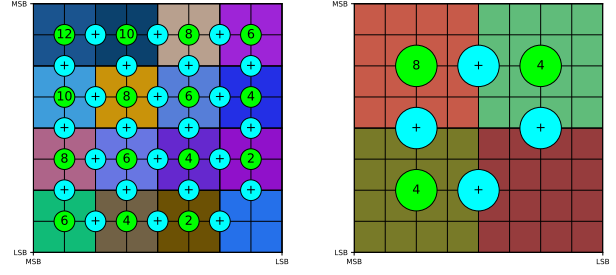
(a) The $n \times n$ multiplier is used for the MSB (The adder tree needs a $2n + 1$ -bit adder).



(b) The $n \times n$ multiplier is used for the LSB (The adder tree needs a $n + 1$ -bit adder).

Fig. 4: $n + 1$ -bit multiplier used in Karatsuba decomposition.

$$ALU \text{ Expression} = \begin{cases} Z + W + X + Y + C_{in} \\ Z - (W + X + Y + C_{in}) \\ -Z + (W + X + Y + C_{in}) \\ \text{not}(Z + W + X + Y + C_{in}) \end{cases} \quad (5)$$



(a) Tiling 8×8 using 2×2 . (b) Tiling 8×8 using 4×4 .
Fig. 5: Different tiling schemes for an 8×8 multiplier.

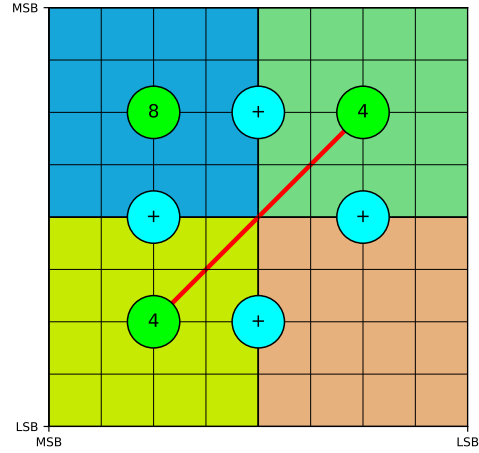


Fig. 6: Karatsuba decomposition of an 8×8 multiplier. The red line connects two multipliers that can be replaced by one.

The DSP multiplier requires proper OPMODE configuration to route the partial products U and V through the X and Y multiplexers, respectively, while simultaneously selecting the appropriate input source (such as PCIN when implementing cascade chaining) for the Z multiplexer. This configuration enables both the fundamental multiplication operation and optional cascading functionality through careful signal routing control.

3) Cascading Two DSP Slices

Two DSP slices can be cascaded to create a 44×18 -bit signed multiplier. Assuming that we have two inputs K of size 44 bits and M of size 18 bits, then Equation 6 can be used.

$$\begin{aligned} K_L &= \{0, K[16 : 0]\} \\ K_H &= K[43 : 17] \\ K &= (K_L + K_H \ll 17) \\ K \times M &= (K_L \times M) + (K_H \times M \ll 17) \end{aligned} \quad (6)$$

To fit within the 48-bit datapath and leverage the built-in 17-bit shift, rewrite the accumulation as in Equation 7.

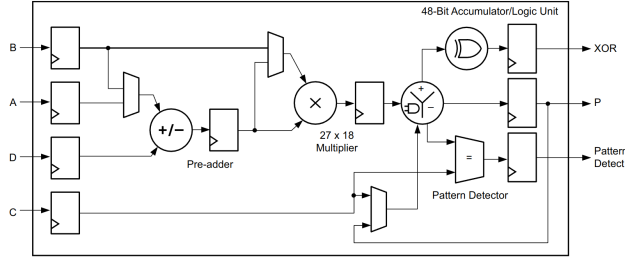


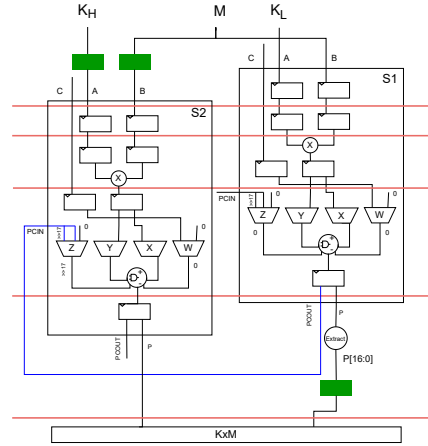
Fig. 7: Simplified DSP48E2 Slice architecture.

$$\begin{aligned}
 P1 &= (K_L \times M) \\
 P2 &= (K_H \times M) \\
 K \times M &= \{(P2 + (P1 \gg 17)), P1[16:0]\}
 \end{aligned} \tag{7}$$

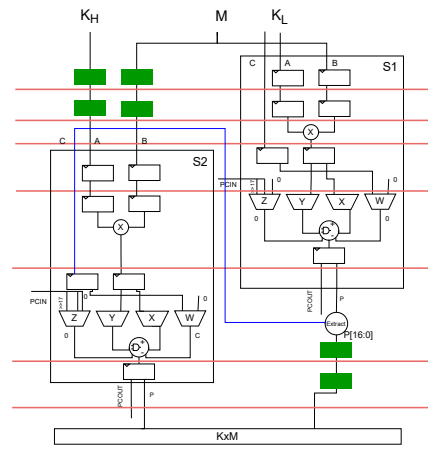
Figure 8 illustrates the cascaded DSP implementation with 4-cycle latency per slice when operating at maximum frequency. The design routes S1's P output (17 LSBs) to global routing while cascading PCOUT to S2's PCIN (blue line in Figure 8a), with S2's OPMODE configured for 17-bit PCIN shifting. Proper signal synchronization requires delaying K_H and M inputs by one clock cycle for S2 synchronization, and the same for P1's 17 LSBs before concatenation with P2 to form the final product, yielding a 5-cycle total latency. The cascade path only supports 0 or 17-bit shifts. For other shift values, a LUT-based shift and sign extension, and the global routing P to C path is used as in Figure 8b. Additional pipeline registers must be inserted to account for this additional cycle, resulting in a longer latency. Figure 8 shows the registers implemented in the logic fabric in green.

D. Signed vs Unsigned Multiplication

When decomposing large multipliers, careful consideration must be given to signed versus unsigned operations. Unsigned multipliers can be directly decomposed into smaller unsigned multipliers with full bit utilization. However, signed decomposition requires special handling; only the most significant portions of each input should use signed multiplication, while all other products must be computed as unsigned operations. This constraint becomes particularly important when using signed base primitives, which require zero-padding for unsigned operations. For instance, a signed 64×64 -bit base multiplier can only scale to a 127×127 -bit multiplier, as the least significant portions need MSB zero-padding. The general formula $(base_size - 1) * 2^n + 1$ governs this relationship, producing valid sizes of 127, 253, 505, and 1009 bits for decomposition levels $n=1$ through 4 when starting from a 64×64 base.



(a) Cascade Path



(b) P to C path

Fig. 8: 44×18 Multiplier using two cascaded DSP blocks with different paths. The red lines represent the synchronization barriers. Green rectangles represent pipeline stages added outside of slices. The blue line distinguishes between the cascade path and the P-to-C path.

E. IP Multipliers

AMD provides a Vivado IP block [43] that can be used to build parametrized multipliers up to 64×64 bits, with options to optimize for frequency or resource utilization, reporting the resulting number of pipeline stages. At sizes larger than 47×47 bits, only the performance-optimal design is offered. While this achieves maximum DSP block frequency, it uses a large number of DSP blocks.

III. LARGE MULTIPLIER DESIGN

Large integer multiplication has been implemented using the different decompositions discussed in Section II. Some work has explored manual decomposition for not-so-small operand sizes, as in [27], [28], [30]. The authors of [22] used an automatic equality saturation framework

along with a cost model to hierarchically construct large multipliers from smaller ones while deciding which decomposition scheme should be used at each level. However, their approach does not consider the underlying architecture of the FPGA, relying on an HLS base multiplier, and thus does not achieve high frequency or area efficiency. Our approach focuses on building an efficient base multiplier using DSP blocks, then hierarchically tiling it up to large sizes while considering the FPGA architecture and DSP block features and while using fast pipelined combinational logic.

A. Building a Base Multiplier

To construct large multipliers efficiently on FPGAs, we employ a base multiplier built using a tiling algorithm inspired by Roy et al. [28]. Their algorithm first finds the closest size that is a linear combination of the two co prime dimensions, $W_1 \times W_2$ of the DSP blocks (i.e. 26×17 for Xilinx DSP48E2). That is, for a desired multiplier size of $M \times M$, it finds integers n and m such that:

$$\bar{M} \geq n \cdot W_1 + m \cdot W_2.$$

The algorithm then starts from the bottom side of the square by placing n tiles of size $W_1 \times W_2$ and m tiles of size $W_2 \times W_1$ in a rectangular grid to cover the area of size $\bar{M} \times \bar{M}$. The above step will fill the boundary area of the square, leaving a smaller square area in the middle uncovered. This middle area is then recursively tiled using the same approach until the entire square is covered. However, this algorithm suffers from several limitations that our work aims to address:

- The algorithm does not always yield a perfect square in the remaining middle area for every n and m values.
- It is not automated to consider the specific architecture of the DSP blocks, such as cascade paths and shift capabilities, which can lead to suboptimal resource utilization and performance.
- The algorithm does not provide flexibility in optimizing for different design goals, such as minimizing LUT usage or latency, which are crucial for various applications.

To address the first problem, we fix one of the parameters to 1, i.e., either $n = 1$ or $m = 1$ to ensure that the remaining middle area is always a perfect square. The proof that this constraint ensures a square remaining area is shown as the following:

TODO: The proof can be removed

a) Setup

Let W_1 and W_2 be positive integers (not necessarily coprime). For positive integers n, m define

$$M = nW_1 + mW_2, \quad k = n + m.$$

We form a square of side length M ; its total area is M^2 .

The tiling algorithm fills in the boundary with tiles using $4(k - 1)$ rectangular tiles of size $W_1 \times W_2$. The total area covered by these tiles is

$$A_{\text{tiles}} = 4(k - 1)W_1W_2.$$

The remaining uncovered region (the “middle area”) is

$$M = M^2 - A_{\text{tiles}} = (nW_1 + mW_2)^2 - 4(n + m - 1)W_1W_2.$$

b) Special cases

Case 1: $n = 1$. Then $M = W_1 + mW_2$ and $k = 1 + m$. Substituting into the expression for M :

$$\begin{aligned} M &= (W_1 + mW_2)^2 - 4mW_1W_2 \\ &= W_1^2 + m^2W_2^2 + 2mW_1W_2 - 4mW_1W_2 \\ &= W_1^2 + m^2W_2^2 - 2mW_1W_2 \\ &= (W_1 - mW_2)^2. \end{aligned}$$

Hence M is a perfect square for any $m \geq 1$.

Case 2: $m = 1$. Then $M = nW_1 + W_2$ and $k = n + 1$. Substituting:

$$\begin{aligned} M &= (nW_1 + W_2)^2 - 4nW_1W_2 \\ &= n^2W_1^2 + W_2^2 + 2nW_1W_2 - 4nW_1W_2 \\ &= n^2W_1^2 + W_2^2 - 2nW_1W_2 \\ &= (nW_1 - W_2)^2. \end{aligned}$$

Again M is a perfect square for any $n \geq 1$.

c) Conclusion

If either $n = 1$ or $m = 1$, the middle area M reduces to $(W_1 - mW_2)^2$ or $(nW_1 - W_2)^2$ respectively, which is always a perfect square. This holds for arbitrary positive integers W_1, W_2, n, m .

Also, the DSP blocks dealing with the MSB can utilize the full signed multiplier range (i.e. 27×18), while those handling only LSBs must use the unsigned range (i.e. 26×17). So we can either have $(n \cdot 26 + m \cdot 17)$ bit unsigned multiplier or $(n \cdot 26 + m \cdot 17 + 1)$ bit signed multiplier. Figure 9 shows the tiling scheme for two different multiplier sizes, 44×44 and 78×78 signed multipliers.

To address the second and third problem, we decouple the mathematical expression of the tiling algorithm from the hardware generation. This allows us to first generate the multiplication terms (tiles) based on the tiling algorithm, and then map these terms to the DSP blocks while considering their specific features, such as cascade paths and shift capabilities. Then, we optimize the mapping process based on the design goals, whether minimizing LUT usage or latency.

Figure 10 illustrates two different grouping strategies for the tiling algorithm. The first strategy (Figure 10a) groups all multiplication terms into a single group, minimizing LUT usage by maximizing the use of connected

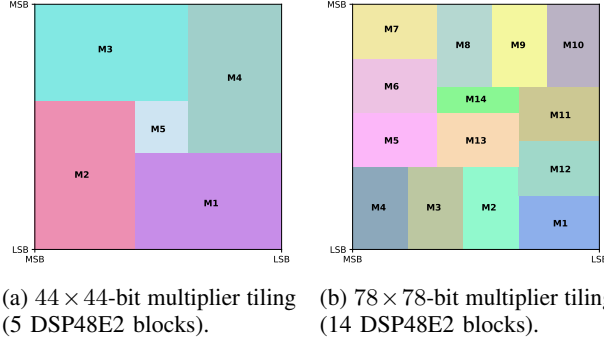
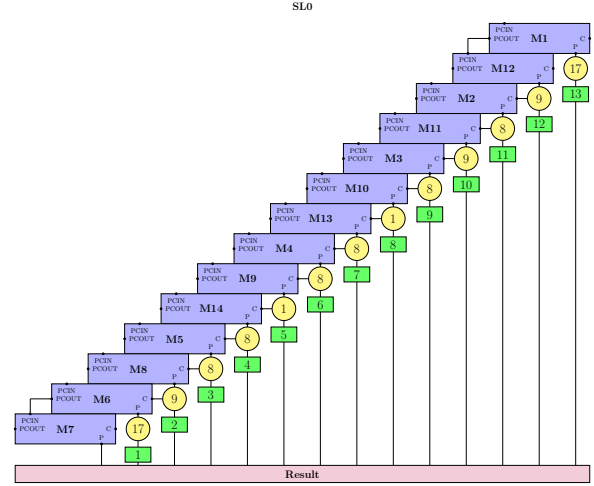


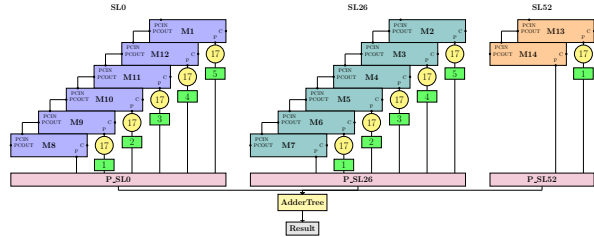
Fig. 9: Tiling schemes for different multiplier sizes.

DSP48E2 blocks without the need for additional logic. The second strategy (Figure 10b) divides the terms into three independent groups that can be computed in parallel, reducing latency at the cost of increased LUT usage used in the adder tree.

Figure 10 provides a visual representation of the two strategies. Green rectangles show pipeline delays for timing synchronization and yellow circles represent extraction with right-shift and 48-bit sign-extension for the next slice's input. The one group strategy starts by sorting terms in ascending order based on their shift size, enabling systematic bit extraction at each stage while propagating the remaining bits to the next slice. This process inherently converts left shifts into right shifts using Equation 6 and 7. All operations are carefully constrained within the 48-bit output limit, preventing unnecessary LUT or DSP overhead to maintain optimal resource utilization. Non-overlapping least significant bits from each multiplier are directly routed to their corresponding positions in the final product, ensuring that additions with sign extension never exceed the 48-bit threshold. With naive ordering, overlapping bits could exceed this limit and thus require wider additions and wasteful resource usage. The required right shift size matches the number of non-overlapping bits between consecutive DSP slices. The implementation uses two distinct data paths selected by shift requirements: the dedicated cascade path handles 17-bit or zero shifts without additional logic, while non-standard shifts utilize the P-to-C path. In the latter case, the shift amount determines both the LSBs routed directly to the final product and the remaining bits' transformation (right-shifted and sign-extended to 48 bits for the next DSP's C input). The design primarily uses P-to-C paths with two exceptions: M1 to M12 and M6 to M7 connections utilize cascade paths. Section II-C3 details the synchronization rules governing cascade path operation. On the other hand, the multi-group strategy divides the multiplication terms into three independent groups, each of which contains terms that are shifted apart by 17 bits. This maximizes the use of cascade paths within each group avoiding the



(a) One group strategy (Minimizing LUTs).



(b) Multi-group strategy (Minimizing latency).

Fig. 10: Different grouping strategies for tile mapping optimization. Green rectangles denote the number of pipeline stages. Yellow circles denote the number of the least significant bits to be extracted from P. Also they denote right shift operation of the P by the same amount and pass it to C if the P to C path is used.

P-to-C path as much as possible, allowing for parallel computation of the groups to minimize overall latency. All groups can be computed in parallel, as shown in Figure 10b. Table I compares the resource usage and latency of different base multipliers built using the two grouping strategies for both 44×44 and 78×78 multipliers. As shown, the one group strategy uses fewer LUTs but has double the latency, while the three groups strategy reduces latency at the cost of slightly increased FF usage. The reason for the increased LUT usage in the 78×78 one group variant is that some LUTs are used to balance the pipeline stages instead of FF registers.

To test the effectiveness of our tiling algorithm and base multiplier design, we compare one of our base multipliers against the AMD Vivado IP multiplier. The AMD Vivado IP block offers a maximum size of 64×64 bits and implemented following the design methodology presented in [43]. Our optimized base multiplier matches

TABLE I: Comparison of different ways to build base multipliers

Primitive Strategy	Size	LUTs	FFs	DSPs	Latency (Cycles)
One group	78	537	1051	14	28
	44	78	323	5	9
Multi-group	78	429	1386	14	14
	44	105	334	5	8

TABLE II: Comparison of 64×64 IP multiplier and our 78×78 base multiplier when tiled to unsigned larger multiplier sizes. SB denotes Schoolbook, Karat. denotes Karatsuba. The wasted DSPs is calculated based on the theoretical minimum calculated using Equation 8.

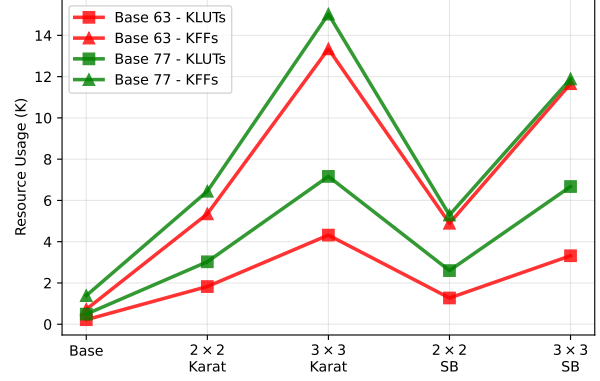
	Tiled Size (Grid)	Scheme	LUTs	FFs	DSPs	Freq. (MHz)	Latency (Cycles)	Wasted DSPs
Base 64	64×64	Base	219	722	16	899	18	6.73
	$126 \times 126 (2 \times 2)$	Karat.	1826	5349	48	933	26	12.08
	$126 \times 126 (2 \times 2)$	SB	1272	4892	64	862	23	28.08
	$189 \times 189 (3 \times 3)$	Karat.	4320	13344	96	803	30	15.18
	$189 \times 189 (3 \times 3)$	SB	3319	11658	144	751	27	63.18
Base 78	78×78	Base	429	1386	14	1026	14	0.24
	$154 \times 154 (2 \times 2)$	Karat.	3026	6444	42	953	28	+11.66 (saved)
	$154 \times 154 (2 \times 2)$	SB	2599	5292	56	870	21	2.34
	$231 \times 231 (3 \times 3)$	Karat.	7165	15031	84	860	37	+36.73 (saved)
	$231 \times 231 (3 \times 3)$	SB	6674	11889	126	795	30	5.27

the AMD IP's maximum frequency while supporting larger inputs, and using 2 fewer DSPs at the expense of 210 more LUTs. For optimal resource allocation, we calculate the theoretical minimum number of 26×17 tiles needed to cover the 64×64 IP, or 78×78 primitive-based multipliers (Equation 8), i.e. excluding the forced waste due to unsigned tile use. While the IP multiplier wastes around 7 DSP slices beyond the theoretical minimum, our primitive-based design only sacrifices a few bits in one slice. Figure 11 compares the resource usage and frequency of our base multiplier against the IP-based design for 64×64 and 78×78 multipliers. Our base multiplier consistently uses fewer DSPs than the IP-based design when scaled to larger sizes, as shown in Figure 11b, while using comparable LUT and FF resources, as shown in Figure 11a. Additionally, our base multiplier achieves higher maximum frequencies across all sizes, as illustrated in Figure 12. This demonstrates the effectiveness of our tiling methodology and base multiplier design in optimizing resource utilization and performance for large integer multiplications on FPGAs.

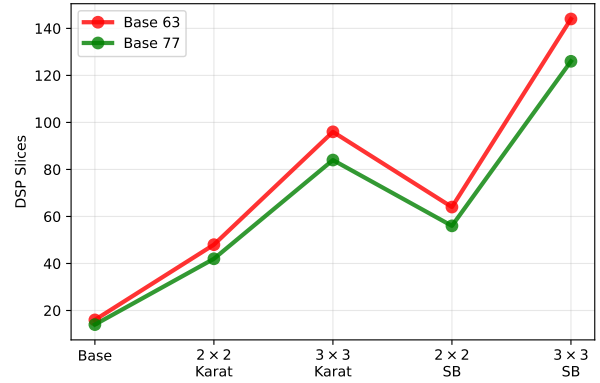
$$\text{optimal_num_DSPs} = \frac{(Size \times Size)}{(26 \times 17)} \quad (8)$$

B. Optimal Base Size Choice

Choosing the optimal base multiplier size is crucial for achieving high performance and area efficiency in large integer multipliers on FPGAs. The base size directly impacts the number of DSP blocks utilized, the depth of the multiplication hierarchy, and the overall latency



(a) LUT and FF usage comparison.



(b) DSP usage comparison.

Fig. 11: Resource usage comparison between Base 64 and Base 78 multipliers when tiled to larger sizes.

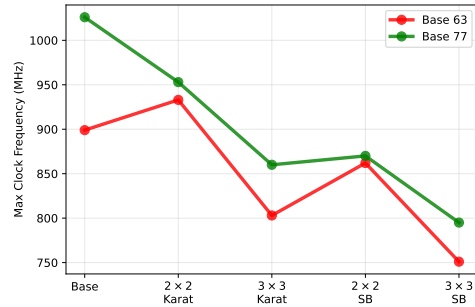


Fig. 12: Maximum frequency comparison between Base 64 and Base 78 multipliers when tiled to larger sizes.

of the design. To determine the optimal base size, we analyze various candidate sizes that can be efficiently mapped to the underlying DSP blocks using the algorithm mentioned in Section III-A. Figure 13 illustrates the DSP waste versus multiplier size over the theoretical minimum calculated by Equation 8 for different base multiplier configurations. From the figure, we observe that base

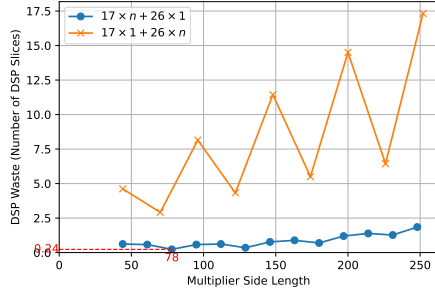


Fig. 13: DSP waste versus multiplier size for different base multiplier configurations.

sizes generated from fixing the greater dimension of the DSP block, i.e. 26 bits in 26×17 DSP48E2, consistently exhibit lower DSP waste across a wide range of multiplier sizes. Among them, the 77×77 base multiplier stands out as it achieves the least DSP waste making it a good fit for tiling it to large sizes.

However, tiling base sizes up has its own challenges apart from using the most efficient base multiplier, i.e. whether or not the base is creating good hierarchical grids that can be further decomposed using Karatsuba multiplication, and how much non-hierarchical area is left to be filled using smaller multipliers. Thus, we need not be restricted to the target size but also consider slightly larger sizes that can be efficiently tiled if it offers better hierarchical nature for area coverage. To illustrate this, if 77×77 is tiled up to the target size is 1024×1024 , the closest size that can be tiled hierarchically is 924×924 leaving two non-hierarchical areas of 100×924 and 1024×100 . However, if we consider 43×43 base multipliers, we can tile to 1032×1032 maximizing the hierarchical area making it easier to decompose using Karatsuba multiplication. For mixed decomposition strategies, different schemes can be applied at different levels, allowing for fine-grained optimization based on the target size structure. To analyze the trade-offs between different base multiplier configurations and their corresponding tiling sizes, we perform a Pareto frontier analysis on various target sizes ranging from 160 to 1024 bits.

The number of DSPs used in base multiplier and when tiling it up hierarchically is known before tiling, so the optimal base size can be chosen based on the target size and the desired decomposition scheme. Here are the equations used to calculate the number of DSPs needed for schoolbook and Karatsuba decomposition schemes when tiling a base multiplier of size $B \times B$ to a target size of $T \times T$:

Given a base multiplier of size $B = nW_1 + mW_2$ (where $W_1 = 26$ and $W_2 = 17$ for DSP48E2), and a target size T that can be expressed as $T = 2^a \times 3^b \times B$,

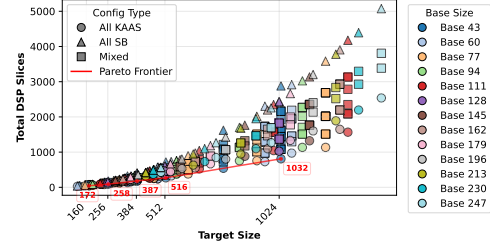


Fig. 14: Pareto frontier for all configurations. The red annotation represents the achieved size using the best base for each target size.

the number of multiplications required at each hierarchical level depends on the grid structure and decomposition scheme used.

For a 2×2 grid:

- Schoolbook decomposition: 4 multiplications
- Karatsuba decomposition: 3 multiplications

For a 3×3 grid:

- Schoolbook decomposition: 9 multiplications
- Karatsuba decomposition: 6 multiplications

Let DSP_{base} denote the number of DSP blocks required for the base multiplier. The total number of DSPs required when tiling hierarchically to size T is:

$$\text{DSP}_{\text{total}} = \text{DSP}_{\text{base}} \times N_{\text{base}} \quad (9)$$

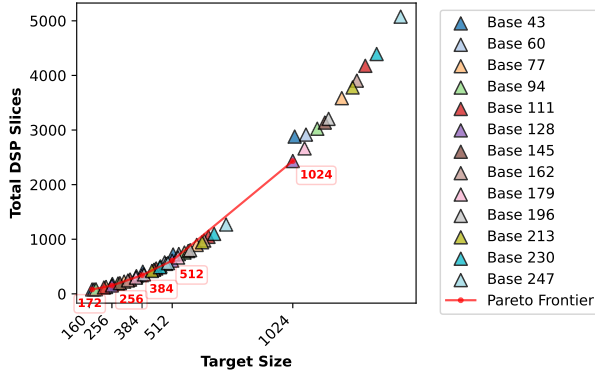
where N_{base} is the total number of base multipliers needed after applying the decomposition scheme across all levels:

$$N_{\text{base}} = \begin{cases} 4^a \times 9^b & \text{(Schoolbook at all levels)} \\ 3^a \times 6^b & \text{(Karatsuba at all levels)} \\ \text{Mixed} & \text{(Combination of schemes)} \end{cases} \quad (10)$$

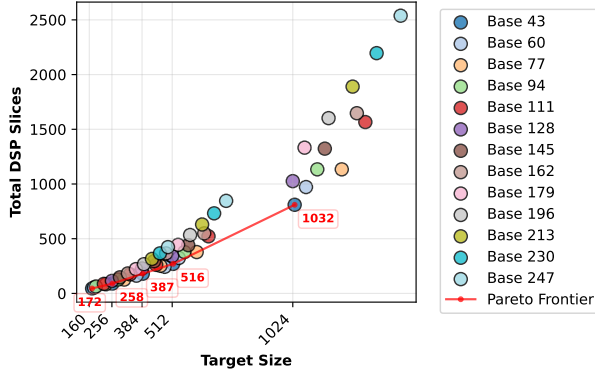
Figure 14 shows the Pareto frontier for all different decomposition schemes at different levels labeling the best base corresponding to each target size. Figure 15 extracts the Pareto frontiers for schoolbook and Karatsuba decomposition schemes separately. From the figures, the 43×43 offers the minimum DSP usage for all target sizes when Karatsuba decomposition is preferred, while 128×128 offers the minimum DSP usage for most target sizes when schoolbook decomposition is used.

C. Tiler

Finding the optimal placement of tiling a large rectangle from smaller rectangles is NP-complete as shown in [23]–[25]. Additionally, general tiling schemes suffer from several limitations when it comes to mapping large multiplications to DSP blocks in FPGAs:



(a) Pareto frontier for all schoolbook configurations.



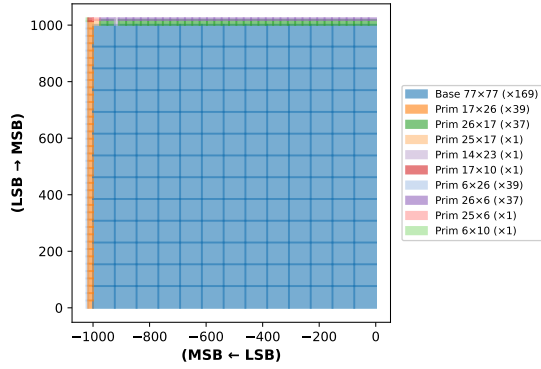
(b) Pareto frontier for all Karatsuba configurations.

Fig. 15: Pareto frontier analysis comparing schoolbook and Karatsuba decomposition schemes.

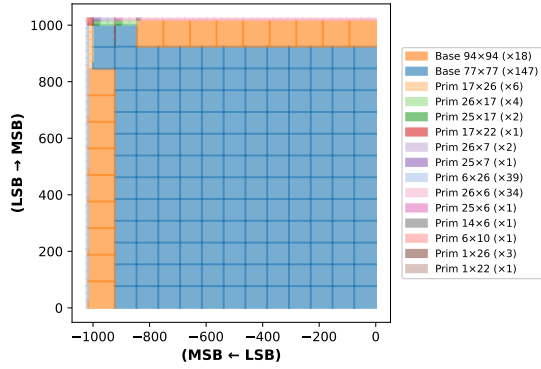
- They do not exploit the hierarchical nature of multiplication algorithms such as Karatsuba multiplication, which can significantly reduce the number of required multiplications.
- They overlook the cascade ability of DSP blocks in non-hierarchical parts of the tiling, which enables for performing certain addition operations in the DSP fabric to achieve high performance and low area utilization.
- They do not consider the fact that the base multiplier sizes that can be efficiently mapped to DSP blocks are limited to certain sizes that are linear combination of the 26 and 17, such as 44×44 , 77×77 , and so on.

Therefore, in this work, we pick the best between an exact heuristic-based tiling algorithm that tiles to the exact multiplication size and another tiling algorithm that does not necessarily tile to the exact size but tries to maximize the hierarchical area that can be covered using the chosen base multiplier size, and thus minimize the non-hierarchical area that needs to be filled using smaller multipliers. Both try to overcome the mentioned limitations and provides an efficient way to map large

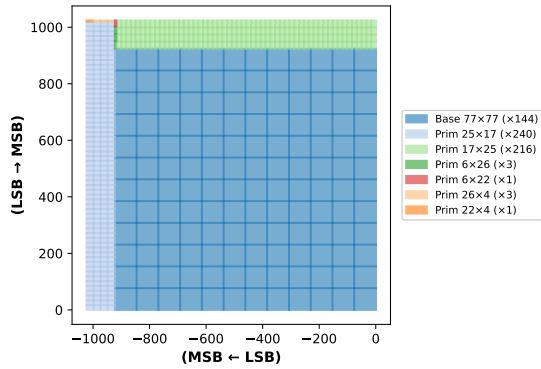
multiplications to DSP blocks in FPGAs. I will consider the example of tiling a 1024×1024 multiplication using 77×77 base multipliers to illustrate the proposed tiling algorithm. The proposed heuristic tiling algorithm works as follows: First, it greedily fills the largest area of the form $2^x \times 3^y \times \text{base_size}$ with the optimal chosen base size, which is 77 in our case. This allows us to leverage both 2×2 and 3×3 hierarchical grids of Karatsuba and Schoolbook multiplication. In the 2×2 grid, each multiplication is decomposed into four smaller multiplications, where one multiplication can be saved using Karatsuba method. On the other hand, 3×3 grid, each multiplication is decomposed into nine smaller multiplications where three multiplications can be saved using Karatsuba method. In the example of 1024×1024 multiplication using 77×77 base multipliers, the largest area that can be covered is 924×924 which is equal to $2^2 \times 3^1 \times 77$. This creates a grid of 12×12 base multipliers that can be tiled in hierarchical fashion, 924 can be decomposed into 2×2 at the first two decomposition levels and 3×3 at the last level. There are three different exact tiling schemes for the remaining areas of 100×924 and 1024×100 using 77×77 base multipliers as shown in Figure 16. First, filling the remaining areas greedily with only the chosen base and DSP48E2-fit multiplications does not provide the optimal solution. Also, filling the area with other base multipliers and DSP48E2-fit multiplications overlooks the cascade ability of the DSP blocks. Therefore, the algorithm solves the remaining areas recursively comparing different placements of other optimal base multipliers i.e. 44×44 and 94×94 in addition to DSP48E2-fit small multipliers, i.e. 23×10 . The comparison is done based on the number of DSP48E2 blocks used in each placement in addition to estimating the number of LUTs and FFs used to combine the outputs of the partial products. Then, it selects the placement that provides the best area and performance trade-off taking into consideration the cascade ability of the DSP blocks. In the example of 1024×1024 multiplication, the remaining areas are 100×924 and 1024×100 . Figure 16 shows the different exact tiling schemes for the 1024×1024 multiplication using 77×77 base multipliers and shows the advantage of the proposed recursive tiling algorithm that takes the cascade ability of the DSP blocks into consideration which uses 464 DSP48E2 blocks while minimizing number of LUTs needed for combining partial products by leveraging the cascade ability. On the other hand, the greedy approach (Figure 16a) uses 507 DSP48E2 blocks and using base multipliers ignoring the cascade ability uses 516 DSP48E2 blocks (Figure 16b), where 21 DSP blocks are needed to perform 94×94 multiplication. Then, the exact tiling algorithm is compared against another tiling algorithm that does not necessarily tile to the exact size but tries to



(a) Greedy Approach



(b) Recursive Approach (No Cascade Consideration)



(c) Recursive Approach (With Cascade Consideration)

Fig. 16: Different tiling schemes for 1024×1024 multiplier. (a) Greedy approach uses 507 DSP48E2 slices for the two remaining blocks. (b) Recursive approach that ignores DSP cascade ability uses 516 DSP48E2 slices. (c) Recursive approach that takes DSP cascade ability into consideration uses 464 DSP48E2 slices.

maximize the hierarchical area that can be covered using the chosen base multiplier size.

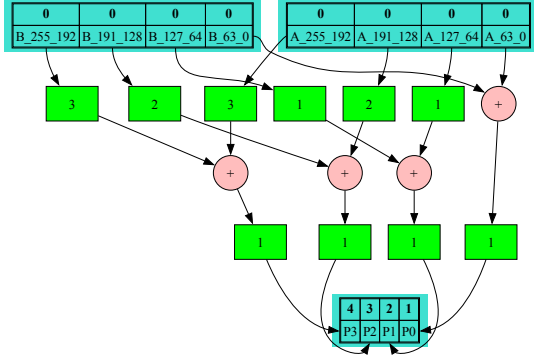
D. Fast Pipelined Combinational Logic

Having established an efficient base multiplier and tiling strategy, we now focus on optimizing the com-

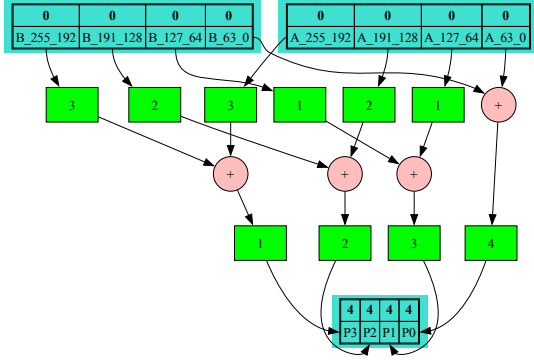
binational logic that interconnects these multipliers in a pipelined architecture. In large integer multipliers, the combinational logic is very wide which can limit the maximum achievable clock frequency. To address this, we propose a fast pipelined combinational logic design that breaks down wide logic paths into smaller segments, allowing for higher clock frequencies while maintaining overall throughput. The key idea is to divide the wide operation into multiple deeply pipelined parts, each handling a portion of the logic. For example, consider a wide addition operation that combines outputs from multiple base multipliers. Instead of performing this addition in a single stage, we can break it down into several smaller additions, each followed by a pipeline register. This approach reduces the critical path delay, enabling higher clock frequencies. After reporting the frequency of multiple different sizes of combinational logic, we observed that the achieved frequency drops as the width of the logic exceeds 64-bits. Choosing the base combinational logic size introduces a trade-off between delay and frequency. To optimize this trade-off, our logic pipelining is configurable, allowing users to select the base logic size based on their specific performance requirements. Moreover, we implement two pipelining strategies: Interleaved Pipelining and LockStep Pipelining. In interleaved pipelining, data flows through multiple combinational modules in an interleaved manner, allowing for continuous data processing and maximizing throughput. In LockStep pipelining, all combinational outputs of a certain module are synchronized before proceeding to the next module. Figures 17a and 17b illustrate these two strategies for a 256-bit multiplier using a 64-bit base. The interleaved approach allows for overlapping operations allowing for different parts of the output to be passed to the next module as soon as they are ready, while the LockStep approach ensures that all parts of the output are ready before moving on, which can simplify timing analysis and control logic.

E. Automating the Whole Flow

To streamline the design process and facilitate rapid exploration of different configurations, we have developed an automated flow that integrates all the components discussed in the previous sections. The flow begins with a user-defined specification including the target multiplier size, the desired decomposition strategy (e.g., Schoolbook, Karatsuba, or mixed), and the base combinational size. Then, leveraging the base multiplier designs and tiling algorithms described in Sections III-A and ??, the tool automatically generates a hierarchical multiplier architecture. The generated architecture incorporates the optimized pipelined combinational logic as outlined in Section III-D to ensure high performance. For each generated multiplier modules and submodules, the tool produces a



(a) Interleaved pipelining for 256-bit multiplier using 64-bit base.



(b) LockStep pipelining for 256-bit multiplier using 64-bit base.

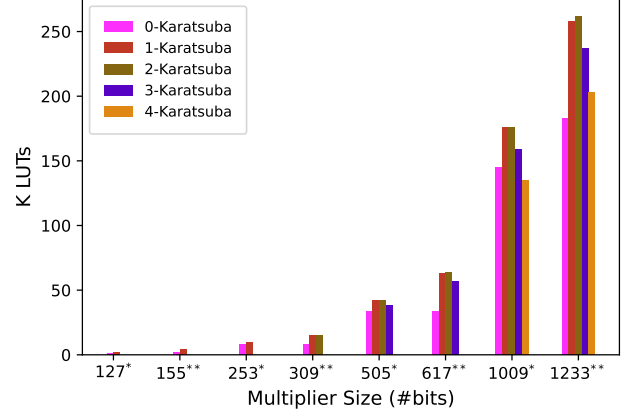
Fig. 17: Comparison of interleaved and LockStep pipelining strategies for large integer multiplication. The green boxes denote pipeline registers, while the cyan boxes represent the input and output parts with a number denoting after how many cycles they are available.

complete Verilog implementation along with testbenches for functional verification. Additionally, it generates Tcl scripts to facilitate synthesis and implementation in FPGA design tools, allowing for quick evaluation of resource utilization and timing performance. Also, for each module, it generates dataflow diagrams that visually represent the mathematical operations and pipeline synchronization points.

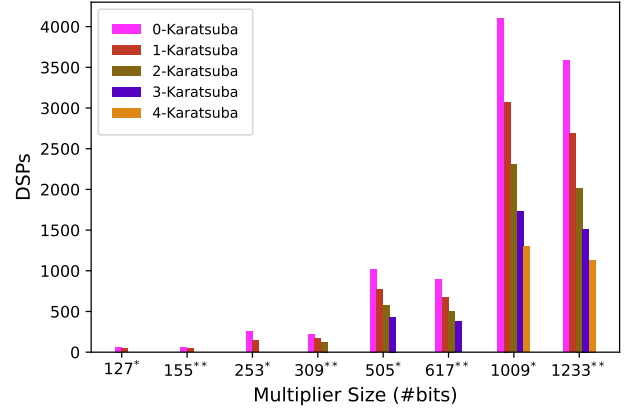
IV. EVALUATION

A. Experiments and Results

We target the AMD Alveo U280 FPGA (XCU280), part xcu280-fsvh2892-2L-e. We tiled both our 64×64 IP base multiplier and our own 78×78 base multiplier hierarchically, varying the number of Karatsuba and



(a) LUT Utilization with Different Decomposition Schemes



(b) DSP Utilization with Different Decomposition Schemes

Fig. 18: Resource Utilization for the IP-based (*) and Primitive-based (**) multipliers, with varying numbers of Karatsuba levels.

Schoolbook decompositions levels as in [22]. Our analysis compares these implementations against previous work across three key metrics: frequency, LUT utilization, and DSP utilization. Due to signed multiplication constraints, we built a 1009-bit multiplier with the IP base and a 1233-bit multiplier with our proposed base, then normalized the results to 1024-bit equivalents for comparison with previous research.

Table III compares our IP-based and primitive-based large multipliers against an HLS-based design and those generated by IMpress in terms of resource utilization and performance. Our IP-based multipliers consistently achieved significantly higher frequencies than both IMpress and HLS implementations across all decomposition schemes while being less efficient in terms of resource utilization because the 64×64 is not so efficient in the first place. Our primitive-based large multipliers achieved higher frequency than the IP-based multipliers in all

TABLE III: Area and performance comparison between all designs. Our IP-based and Primitive-based multipliers achieved much higher frequency than other approaches, with the Primitive being more resource-efficient. * This column calculates the wasted DSPs by comparing the actual implementation against a scaled theoretical optimum.

	#Karat. Levels	LUTs	FFs	DSPs	Freq (MHz)	Latency (Cycles)	#Wasted DSPs *
HLS	3	71893	N/A	1476	256	17	475
	4	88986	N/A	1296	305	21	545
	5	126193	N/A	972	302	21	409
	6	165618	N/A	729	298	24	307
Impress1	N/A	150400	N/A	810	250	23	247
Impress2 [22]	N/A	129338	N/A	945	295	22	382
Primitive	4	143937	183155	837	528	229	87
	3	218891	299941	1131	453	214	130
	2	288082	381662	1550	438	193	216
	1	50936	68165	1709	442	167	70
	0	119262	139371	2471	428	136	99
IP	4	138860	66408	1328	445	220	578
	3	164508	27374	1760	440	211	759
	2	201416	91435	2432	300	196	1097
	1	173580	872846	3000	246	176	1221
	0	149237	247251	4219	253	125	1846

TABLE IV: Comparison of exact and non-exact tiling for different multiplier sizes

Mult Size	Base	LUTs	FFs	DSPs	Max Freq. (MHz)	Latency (Cycles)
160	77	3609	7450	76	783	28
163	prim	3461	6112	61	830	35
172	43	5141	9202	45	862	30
256	43	—	—	—	—	44
256	77	8473	17718	114	801	39
258	43	10747	20219	90	792	36
512	43	—	—	—	—	74
512	77	—	—	—	—	74
516	43	38131	68870	270	629	60
1024	77	123479	207794	1220	484	123
1032	43	128282	221612	810	458	100

different configurations in addition to being resource efficient, utilizing fewer DSPs, but slightly more LUTs. Impress achieves better overall performance than the HLS design in terms of frequency and area utilization. Our primitive-based multiplier achieved at least 45%, and up to 73% better frequency than the best-performing multiplier in the HLS and Impress implementations.

Figure 19 compares the best-performing models in different implementations in terms of the maximum achieved frequency against their LUTs and DSP usage. The results indicate distinct performance-resource tradeoffs: Impress2 has the least LUT utilization, but has the second-highest DSP utilization while not achieving high-frequency. On the other hand, the full Karatsuba HLS implementation uses the least number of DSPs, but uses the largest number of LUTs and achieves the same low frequency as Impress. Our IP-based approach achieved a much higher frequency than other implementations using LUTs efficiently, but also using more DSP blocks than other implementations. Our primitive-based multiplier with 4 levels of Karatsuba decomposition (the grey circle) achieved by far the highest frequency while using an LUTs and DSPs within the range

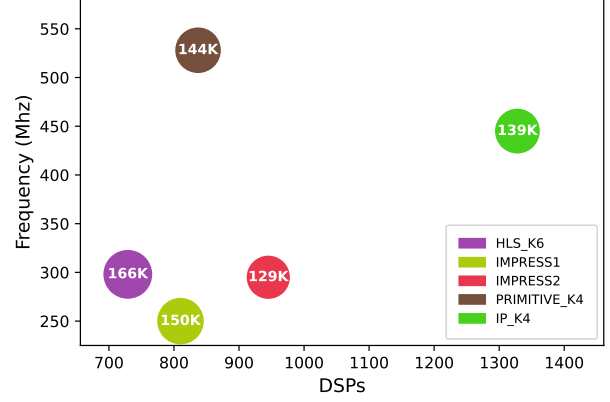


Fig. 19: Area and Performance comparison of different implementations. The size of each circle and the number inside denotes LUT utilization.

of other implementations. Though they do achieve higher frequency, and hence throughput, our implementations do suffer from longer latency due to the deep pipelining required to map efficiently to the DSP blocks.

The DSP efficiency of each implementation is evaluated by comparing its utilization against a scaled theoretical optimum calculated from Equation 8. The base square size is 16 bits for HLS, 32 for Impress, and 64 for our implementation. This optimal reference is calculated by scaling the baseline DSP counts, where each Karatsuba decomposition level introduces 3 multipliers and each Schoolbook level 4 multipliers, that is $(base_opt_DSP_count * 3^{\#Karatsuba} * 4^{\#Schoolbook})$. IP-based implementations achieve the poorest resource efficiency with the highest DSP waste, primarily due to the inherent inefficiency of the 64×64 IP base multiplier. In contrast, our primitive-based approach achieves maximal efficiency with minimal DSP waste. Intermediate implementations like HLS and Impress show significantly higher DSP waste compared to our primitive-based design, though still outperform the IP-based approach. This shows that coupling the tiling automation with an efficient hand-crafted base primitive is key to maximizing throughput and resource efficiency.

V. RELATED WORK

Authors in [44] developed an automated tool for mapping arithmetic expressions to DSP blocks while preserving high operating frequency. They extended this work in [45] through multipumping techniques to enable efficient resource sharing. While these methods demonstrate effective DSP utilization, both approaches rely on fixed template databases that constrain their generalizability. Additionally, neither solution addresses the challenges of large integer multiplication. Authors in [22] presented Impress, a framework that utilizes rewrite rules,

E-graphs, and equality saturation to decompose a large multiplier into a hierarchy of smaller ones implemented using HLS. However, the base HLS multipliers do not achieve high frequency. Extending general decomposition schemes to be architecture-aware has been explored in previous work. In [28], the authors proposed a tiling algorithm to create a 58×58 -bit multiplier that was previously obtained in [27] and to generalize for arbitrary wordlength multiplier and for arbitrary DSP multiplier width. The algorithm represents the input width, or a close number, as a linear combination of DSP multiplier input sizes. Then, the algorithm places rectangles on the boundary, then recursively solves the reduced square problem in the middle. However, it is limited to Schoolbook decomposition and square multiplier inputs, and it does not scale well for large numbers, because the reduced problem is not always a square area.

In [30], the authors implemented a rectangular multiplier with Karatsuba decomposition. They replaced subtractions in Equation 3 with additions to effectively use the post-adder of the DSP slice. However, this work lacks a scalable, generalizable algorithm for bigger operand sizes. Authors in [29] proposed a binary ILP formulation for integer multiplication, but it takes several days to compute sizes beyond 64×64 bits. Then, in [31], they extended their previous ILP approach to adopt Karatsuba decomposition as a special tile in the ILP problem along with some greedy and beam search heuristics. Both run at a low frequency and have a limited max operand size of 64 bits.

VI. CONCLUSIONS AND FUTURE WORK

Since large integer multiplication is needed by many applications, it must be implemented in a way that maximizes performance while also using resources efficiently. In this work, we proposed an IP-based base multiplier as well as a new hand-crafted design of a base multiplier that can be used to form large integer multipliers, optimized for the AMD DSP48E2 primitive. We created a tool that automatically generates the RTL code for performing different decomposition schemes of large integer multiplication down to these two base multipliers. Both designs achieve high frequencies above 400MHz. The proposed Primitive-based approach achieved around 73% higher frequency than other approaches while using around 13% fewer DSP blocks at the expense of 5% more LUTs.

In this work, we hand-crafted the base multiplier around the DSP48E2 primitive. In the future, we plan to leverage equality saturation and rewrite rules to provide a tool that automatically builds the large integer multiplication, whether square or rectangle, directly from arbitrary DSP primitives. It would also be possible to extend this to other functions beyond multiplication. Finally, some strategies

for mitigating the cross-SLR overhead, including another level of hierarchical pipelining could be explored.

VII. REFERENCES

- [1] D. Patel, B. Patel, J. Vasa, and M. Patel, "A comparison of the key size and security level of the ECC and RSA algorithms with a focus on cloud/fog computing," in *ICT with Intelligent Applications*, 2023, pp. 43–53.
- [2] Z. Ji, J. Zhao, Z. Zhang, J. Xu, S. Yan, and L. Ju, "A compiler-like framework for optimizing cryptographic big integer multiplication on GPUs," in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2024, pp. 380–392.
- [3] S. Memik, A. Katsaggelos, and M. Sarrafzadeh, "Analysis and fpga implementation of image restoration under resource constraints," *IEEE Transactions on Computers*, pp. 390–399, 2003.
- [4] P. Zemcik, "Hardware acceleration of graphics and imaging algorithms using fpgas," in *Proceedings of Spring Conference on Computer Graphics*, 2002, pp. 25–32.
- [5] A. G. Ye and D. M. Lewis, "Procedural texture mapping on fpgas," in *Proceedings of ACM/SIGDA seventh international symposium on Field programmable gate arrays*, 1999, pp. 112–120.
- [6] J. Isoaho, J. Pasanen, O. Vainio, and H. Tenhunen, "Dsp system integration and prototyping with fpgas," *J. VLSI Signal Process. Syst.*, pp. 155–172, 1993.
- [7] S. Demidenko, D. Johnson, K. Gribbon, and D. Bailey, *Implementing Signal Processing Algorithms in FPGAs: Digital Spectral Warping*, 2002.
- [8] R. Tessier and W. Burleson, "Reconfigurable computing for digital signal processing: A survey," *Journal of VLSI signal processing systems for signal, image and video technology*, pp. 7–27, 2001.
- [9] G. C. Chow, K. Eguro, W. Luk, and P. Leong, "A karatsuba-based montgomery multiplier," in *2010 International Conference on Field Programmable Logic and Applications*, 2010, pp. 434–437.
- [10] D. D. Chen, G. X. Yao, R. C. Cheung, D. Pao, and Ç. K. Koç, "Parameter space for the architecture of fft-based montgomery modular multiplication," *IEEE Transactions on Computers*, vol. 65, no. 1, pp. 147–160, 2016.
- [11] M. M. Islam, M. S. Hossain, MD. Shahjalal, M. K. Hasan, and Y. M. Jang, "Area-time efficient hardware implementation of modular multiplication for elliptic curve cryptography," *IEEE Access*, vol. 8, pp. 73 898–73 906, 2020.
- [12] M. A. Mehrabi, C. Doche, and A. Jolfaei, "Elliptic curve cryptography point multiplication core for hardware security module," *IEEE Transactions on Computers*, vol. 69, no. 11, pp. 1707–1718, 2020.

- [13] T. Ye, S. R. Kuppannagari, R. Kannan, and V. K. Prasanna, "Performance modeling and fpga acceleration of homomorphic encrypted convolution," in *2021 31st International Conference on Field-Programmable Logic and Applications (FPL)*, 2021, pp. 115–121.
- [14] A. C. Mert, E. Öztürk, and E. Savaş, "Design and implementation of encryption/decryption architectures for bfv homomorphic encryption scheme," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 28, no. 2, pp. 353–362, 2020.
- [15] P. G. Comba, "Exponentiation cryptosystems on the IBM PC," *IBM Systems Journal*, pp. 526–538, 1990.
- [16] A. Karatsuba and Y. Ofman, "Multiplication of multidigit numbers on automata," *Soviet Physics Doklady*, vol. 7, p. 595, 1962.
- [17] C. Yap and C. Li, "Quickmul: Practical fft-based integer multiplication," 2000.
- [18] A. Emerencia, "Multiplying huge integers using fourier transforms," 2007.
- [19] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex fourier series," *Mathematics of Computation*, pp. 297–301, 1965.
- [20] A. Schönhage and V. Strassen, "Schnelle multiplikation großer zahlen," *Computing*, pp. 281–292, 1971.
- [21] N. Emmart and C. C. Weems, "High precision integer multiplication with a gpu using strassen's algorithm with multiple fft sizes," *Parallel Processing Letters*, pp. 359–375, 2011.
- [22] E. Ustun, I. San, J. Yin, C. Yu, and Z. Zhang, "IMpress: Large integer multiplication expression rewriting for FPGA HLS," in *Field-Programmable Custom Computing Machines (FCCM)*, 2022, pp. 1–10.
- [23] Beaumont, Boudet, Rastello, and Robert, "Partitioning a square into rectangles: NP-completeness and approximation algorithms," *Algorithmica*, p. 217–239, 2002.
- [24] L. A. Levin, "Problems, complete in "average" instance," in *ACM Symposium on Theory of Computing*, 1984.
- [25] —, "Average case complete problems," *SIAM Journal on Computing*, pp. 285–286, 1986.
- [26] M. Kumm, J. Kappauf, M. Istoan, and P. Zipf, "Resource optimal design of large multipliers for fpgas," in *IEEE Symposium on Computer Arithmetic (ARITH)*, 2017, pp. 131–138.
- [27] F. Dinechin and B. Pasca, "Large multipliers with less DSP blocks," in *Field Programmable Logic and Applications (FPL)*, 2009.
- [28] D. B. Roy, D. Mukhopadhyay, M. Izumi, and J. Takahashi, "Tile before multiplication: An efficient strategy to optimize DSP multiplier for accelerating prime field ecc for NIST curves," in *Design Automation Conference (DAC)*, 2014, pp. 1–6.
- [29] M. Kumm, J. Kappauf, M. Istoan, and P. Zipf, "Resource optimal design of large multipliers for FPGAs," in *IEEE Computer Arithmetic (ARITH)*, 2017, pp. 131–138.
- [30] M. Kumm, O. Gustafsson, F. De Dinechin, J. Kappauf, and P. Zipf, "Karatsuba with rectangular multipliers for FPGAs," in *Computer Arithmetic (ARITH)*, 2018, pp. 13–20.
- [31] A. Böttcher, K. Kullmann, and M. Kumm, "Heuristics for the design of large multipliers for FPGAs," in *IEEE Computer Arithmetic (ARITH)*, 2020, pp. 17–24.
- [32] Xilinx, Inc., "Virtex-5 FPGA user guide (UG190)," 2009.
- [33] —, "Virtex-6 family overview (DS150)," p. 1, 2015.
- [34] —, "Ultrascale architecture and product data sheet: Overview (DS890)," p. 44, 2024.
- [35] —. (2022) Versal ACAP DSP engine architecture manual (AM004).
- [36] —, "DSP: Designing for optimal results. high-performance DSP using virtex-4 FPGAs (UG073)," 2005.
- [37] T. Hussain, S. Amin, U. Zabit, F. Kamran, O. D. Bernal, and T. Bosch, "A high performance real-time FPGA-Based interferometry sensor architecture," in *International Conference on Frontiers of Information Technology (FIT)*, 2016, pp. 130–135.
- [38] D. Wang, K. Xu, J. Guo, and S. Ghiasi, "DSP-Efficient hardware acceleration of convolutional neural network inference on FPGAs," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (IEEE TCAD)*, pp. 4867–4880, 2020.
- [39] L. Ioannou and S. A. Fahmy, "Streaming overlay architecture for lightweight LSTM computation on FPGA SoCs," *ACM Transactions on Reconfigurable Technology and Systems*, vol. 16, no. 1, pp. 8:1–8:26, 2022.
- [40] P. H. W. Leong, "Recent trends in FPGA architectures and applications," in *IEEE International Symposium on Electronic Design, Test and Applications*, 2008, pp. 137–141.
- [41] M. Langhammer and B. Pasca, "Floating-point DSP block architecture for FPGAs," in *ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*, 2015, pp. 117–125.
- [42] Xilinx, Inc., "Ultrascale architecture DSP slice user guide (UG579)," 2021.
- [43] —, "Multiplier v12.0 LogiCORE IP product guide (PG108)," 2015.
- [44] B. Ronak and S. A. Fahmy, "Mapping for maximum

- performance on FPGA DSP blocks,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, pp. 573–585, 2016.
- [45] —, “Multipumping flexible DSP blocks for resource reduction on xilinx FPGAs,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, pp. 1471–1482, 2017.